University of Michigan - Dearborn
CIS 310
Assignment 3

Aaron Amano
amanoac@umich.edu
April 20, 2026

Project Files

https://github.com/aaronamano/cis310-assignment-3

Test Results

File Tested	Test result
8-bit-full-adder.dig	No test data found.
8-bit-memory.dig	No test data found.
8-bit-register.dig	No test data found.
ALU.dig	No test data found.
control-unit.dig	No test data found.
IR.dig	No test data found.
my-8-bit-Processor.dig	70 test rows passed
PC.dig	No test data found.
registerfile.dig	No test data found.

	clk	Program_Counter	InstrMem_input	InstrMem_ctrl	IR_output	IR_ctrl	Write_Back	ALU_output
L78	0	1	A8	1	0	0	0	0
L79;n=0	0	1	0	0	0	0	0	0
L79;n=1	0	1	0	0	0	1	0	0
L79;n=2	0	1	0	0	A8	0	1	1
L88	0	2	A8	1	A8	0	0	2
L89;n=0	0	2	0	0	A8	0	0	2
L89;n=1	0	2	0	0	A8	1	0	2
L89;n=2	0	2	0	0	A8	0	1	2
L91	0	3	A8	1	A8	0	0	3
L92;n=0	0	3	0	0	A8	0	0	3
L92;n=1	0	3	0	0	A8	1	0	3
L92;n=2	0	3	0	0	A8	0	1	3
L94	0	4	A8	1	A8	0	0	4
L95;n=0	0	4	0	0	A8	0	0	4
L95;n=1	0	4	0	0	A8	1	0	4
L95;n=2	0	4	0	0	A8	0	1	4
L111	0	5	B0	1	A8	0	0	5
L112;n=0	0	5	0	0	A8	0	0	5
L112;n=1	0	5	0	0	A8	1	0	5
L112;n=2	0	5	0	0	B0	0	1	1
L115	0	6	B0	1	B0	0	0	2
L116;n=0	0	6	0	0	B0	0	0	2
L116;n=1	0	6	0	0	B0	1	0	2
L116;n=2	0	6	0	0	B0	0	1	2
L134	0	7	6C	1	B0	0	0	3
L135;n=0	0	7	0	0	B0	0	0	3
L135;n=1	0	7	0	0	B0	1	0	3
L135;n=2	0	7	0	0	6C	0	1	0
L142;n=0	0	0	0	0	6C	0	0	0
L142;n=1	0	0	0	0	6C	0	0	0
L152;n=0	0	0	0	0	6C	1	0	0
L152;n=1	0	0	0	0	A8	0	1	1
L152;n=2	0	1	0	0	A8	0	0	2
L152;n=3	0	1	0	0	A8	0	0	2
L152;n=4	0	1	0	0	A8	1	0	2
L152;n=5	0	1	0	0	A8	0	1	2
L152;n=6	0	2	0	0	A8	0	0	3
L152;n=7	0	2	0	0	A8	0	0	3
L152;n=8	0	2	0	0	A8	1	0	3
L152;n=9	0	2	0	0	A8	0	1	3
L152;n=10	0	3	0	0	A8	0	0	4
L152;n=11	0	3	0	0	A8	0	0	4
L152;n=12	0	3	0	0	A8	1	0	4
L152;n=13	0	3	0	0	A8	0	1	4
L152;n=14	0	4	0	0	A8	0	0	5
L152;n=15	0	4	0	0	A8	0	0	5
L152;n=16	0	4	0	0	A8	1	0	5
L152;n=17	0	4	0	0	B0	0	1	3
L152;n=18	0	5	0	0	B0	0	0	4
L152;n=19	0	5	0	0	B0	0	0	4
L152;n=20	0	5	0	0	B0	1	0	4
L152;n=21	0	5	0	0	B0	0	1	4
L152;n=22	0	6	0	0	B0	0	0	5
L152;n=23	0	6	0	0	B0	0	0	5
L152;n=24	0	6	0	0	B0	1	0	5
L152;n=25	0	6	0	0	6C	0	1	0
L152;n=26	0	7	0	0	6C	0	0	0
L152;n=27	0	7	0	0	6C	0	0	0
L152;n=28	0	7	0	0	6C	1	0	0
L152;n=29	0	7	0	0	6C	0	1	0
L152;n=30	0	0	0	0	6C	0	0	0
L152;n=31	0	0	0	0	6C	0	0	0
L152;n=32	0	0	0	0	6C	1	0	0
L152;n=33	0	0	0	0	A8	0	1	1
L152;n=34	0	1	0	0	A8	0	0	2
L152;n=35	0	1	0	0	A8	0	0	2
L152;n=36	0	1	0	0	A8	1	0	2
L152;n=37	0	1	0	0	A8	0	1	2
L152;n=38	0	2	0	0	A8	0	0	3
L152;n=39	0	2	0	0	A8	0	0	3

How the Processor Operates

The instruction, data, and control type are inputted in 8-bit memory and are stored so it can be retrieved and processed. Next, those instructions are retrieved from the memory where the opcode and operands are extracted. Afterwards, those instructions are executed in the ALU and Register File through the opcode and operands it extracted as well as the data.

1. **Fetch Cycle**
   The Program Counter stores the address of the next instruction, which is sent to the 8-bit-memory. Afterwards the 8-bit-memory receives the address from the PC and gets the instruction from the address. Afterwards the memory sends and loads the instruction into the Instruction Register, so it can be stored and be processed later.

2. **Decode Cycle**
   The Instruction Register extracts the opcode, operands, and other data, which is sent to the control unit. The Control Unit decodes the information and generates the control signals to determine which registers need to be read and what operations/processes need to be performed. The Register File reads the source registers depending on what the registers Instruction Register referred to, and the Register File inputs the register values into the ALU

3. **Execute Cycle**
   The ALU performs the arithmetic and logical operations based on the data it's given. Then, the result is written back into the register file and stored there. Afterwards the program counter is incremented and the next address is stored so that it can be fetched, decoded, and executed again.

FSM Implementation

```
RESET
  │
  ▼
┌──────────────┐
│ FETCH        │ ← PC → Memory, load IR
│ (State 0)    │
└──────┬───────┘
  │
  ▼
┌──────────────┐
│ DECODE       │ ← Control unit generates
│ (State 1)    │   register read & ALU op
└──────┬───────┘
  │
  ▼
┌──────────────┐
│ EXECUTE      │ ← ALU computes, result
│ (State 2)    │   written back, PC updated
└──────────────┘
  │
  └──────────► back to FETCH
```